

Automated Network Request Management System

1. INTRODUCTION

1.1 Project Overview

The Automated Network Request Management System is an enterprise-grade cloud solution developed to replace slow, error-prone manual email interactions and paper-based tracking during workplace transitions. Built on low-code architectural frameworks, this system provides a unified interface for employees to request internal network setups, bandwidth re-allocations, and technical equipment relocation tracking with zero friction.

1.2 Purpose

The purpose of this project is to eliminate operational downtime caused by administrative bottlenecks when departments move spaces. By mapping precise data-capture variables on the frontend and pairing them with automated cloud orchestration paths, this system cuts down request fulfillment cycles, protects institutional data integrity, and establishes completely transparent transaction logs for system administrators.

2. IDEATION PHASE

2.1 Problem Statement

Manual enterprise administrative workflows are plagued by disjointed communications, transcription errors during desk swaps, and zero visibility into active processing cycles. When employees change physical office footprints, setting up their hardware networking profiles requires disjointed coordination across facility departments, network teams, and managers. This friction causes extensive delays, human data entry mistakes, and direct productivity losses for the organization.

2.2 Empathy Map Canvas

To capture customer realities, an individual empathy analysis was executed across core user types:

- **User Says:** "I have no idea where my network setup ticket is stuck or when my port will be activated."
- **User Thinks:** "This administrative delay is making me look unproductive to my manager during my desk move."
- **User Does:** Sends constant follow-up emails, creating duplicate communication noise for IT teams.
- **User Feels:** Frustrated by the total lack of clarity and operational responsiveness.

2.3 Brainstorming

The ideation process isolated three core functional requirements needed to solve this problem:

1. **Unified Intake Portal:** A single portal catalog view using smart dynamic form fields to capture perfect parameters on day one.
2. **Decoupled Workflow Orchestration:** A backend script engine to automatically route requests to managers without manual sorting.
3. **Persistent Structural Ledger:** Directing all verified and approved entries straight into an isolated, secure enterprise database table (u_network_database).

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The user journey tracking covers four main operational milestones:

1. **Discovery:** The user accesses the centralized corporate portal page.
2. **Configuration:** The form extracts profile data automatically, and fields adjust dynamically based on the request type.
3. **Approval Pending:** The user receives a tracking reference badge while an automated notification alerts the manager.
4. **Fulfillment:** The database ledger automatically updates with clean parameters, and the user receives an "Order Placed" completion receipt.

3.2 Solution Requirement

- **Functional Requirements (FR):** Automated identity parsing, dynamic layout updates based on selected options, automated email generation, and validated data persistence checks.
- **Non-Functional Requirements (NFR):** The system must achieve sub-second form rendering speeds and secure cell isolation using Access Control Lists (ACLs).

3.3 Data Flow Diagram

The structural data flow maps out how transactions flow through the application layers:

[User Portal Input] —> (Client-Side Logic) —> [Cloud Flow Orchestration]

[u_network_database] <— (Data Sanity Script) <— [Supervisor Gate]

3.4 Technology Stack

- **Frontend Platform:** Low-Code Service Portal Engine.
- **Process Automation:** Cloud Workflow Studio (Flow Designer Logic Block Engine).
- **Backend Database Matrix:** Relational Custom Data Schema Storage (u_network_database).

4. PROJECT DESIGN

4.1 Problem Solution Fit

The design resolves each identified business problem with a specific technical feature: manual transcription errors are stopped by user-context parsing; tracking anxieties are eliminated by live status tracking badges; and long routing delays are cleared by automated approval task lines.

4.2 Proposed Solution

The system introduces an automated self-service portal connected to a multi-stage approval engine. This setup decouples front-end user actions from the backend database layer, ensuring great scalability and keeping the platform highly stable.

4.3 Solution Architecture

The architecture runs through an optimized multi-tier pipeline to securely process, audit, and commit incoming network provisioning transactions:

1. PORTAL SUBMISSION
2. 2. FLOW ORCHESTRATION ENGINE
3. 3. MULTI-STAGE APPROVAL GATE
4. 4. SECURE INJECTION & DATABASE WRITE

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Using Agile Scrum guidelines for an individual contributor, the project implementation was executed across two distinct 10-day sprints. Tasks were sized using the standard Fibonacci point scale (1 to 5) to accurately measure required development effort:

Sprint-1: UI Logic & Intake Architecture (7 Story Points)

- **USN-1 [2 Pts]:** Automate profile extraction using client-side session variables.
- **USN-2 [3 Pts]:** Configure dynamic visibility rules to display room coordinates based on request type.
- **USN-3 [2 Pts]:** Build the core frontend data handoff to the backend engine.

Sprint-2: Governance, Security & Ledger Commits (12 Story Points)

- **USN-4 [2 Pts]:** Set up automated manager routing queues and email alert rules.
- **USN-5 [5 Pts]:** Write custom server-side injection scripts to populate u_network_database.
- **USN-6 [5 Pts]:** Build an administrative operations dashboard with live metric charts.

Velocity Metric Summary:

- **Total Points Delivered:** 19 Story Points completed over 2 Sprints.
- **Individual Development Pace:** Established an average baseline execution rate of **9.5 Story Points per Sprint**.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

User Acceptance Testing (UAT) and functional validations were run across all system components. Testing focused on evaluating form response times, script execution speeds, and data write security checks.

Defect Resolution Tracking Matrix:

- **Critical Issues (Severity 1):** 8 bugs were successfully fixed and verified. These primarily involved resolving infinite loop conditions in the workflow approval lines and addressing column mapping conflicts during ledger injection.
- **Minor/Cosmetic Issues:** 27 styling layout anomalies were resolved to ensure clean interface rendering across different screen sizes.

Test Suite Pass-Rate Matrix:

- **Service Portal Form UI:** 22 Cases Executed — **100% Pass Rate**
- **Flow Designer Orchestration:** 18 Cases Executed — **100% Pass Rate**
- **Access Control Lists (ACL Security):** 12 Cases Executed — **100% Pass Rate**
- **Database Ledger Integrity:** 15 Cases Executed — **100% Pass Rate**
- **Exception Handling Logic:** 9 Cases Executed — **100% Pass Rate**
- **Total Automated Test Volume: 76 Test Cases Verified with Zero Open Failures.**

7. RESULTS

7.1 Output Screenshots

Network Task Tables				
Assigned to	Assignment Group	Requested for	Task Number	Work Status
Abel Tuter	Analytics Settings Managers	Abel Tuter	REQ0010002	In Progress

8. ADVANTAGES & DISADVANTAGES

Advantages:

- **Zero Admin Overhead:** Automated request routing eliminates the need for manual sorting or data re-entry.
- **Strong Structural Security:** Server-side verification scripts block incomplete parameters or unauthorized database writes.
- **Clear Operational Status:** Live tracking badgs show users exactly where their request sits in the processing queue.

Disadvantages:

- **Platform Dependency:** System availability is tied directly to the uptime of the hosting cloud platform.
- **Initial Setup Time:** Building out robust field verification scripts require additional development time during early deployment phases.

9. CONCLUSION

The Automated Network Request Management System successfully achieves all its primary engineering objectives. Transitioning administrative workflows to an automated, low-code cloud architecture completely removes the operational delays, communication blackholes, and data errors common to manual systems. Testing confirms that the application provides a highly responsive frontend user experience paired with a secure, dependable backend persistence ledger.

10. FUTURE SCOPE

Future updates will focus on incorporating native **Machine Learning classification tools** to analyze past ticket descriptions. This will allow the system to automatically assign high-priority network pathways based on corporate roles, dynamically allocate network resources, and predict equipment provisioning timelines with advanced modeling analytics.

11. APPENDIX

GitHub & Project Demo Link:

- **Repository:** <https://github.com/ruthvicksai-dev/Automated-Network-Request-Management-in-ServiceNow>